



## Context: I'm competing in an "AI Master" hiring challenge (G4, a Brazilian

exec-ed company). Task: act as the AI lead for a customer-support operation — diagnose bottlenecks, propose AI automation, ship a working prototype — using two public Kaggle datasets. Submissions are judged against an LLM baseline (they pre-ran the brief through Claude/GPT/Gemini) and must "substantially beat" it. ~80 candidates competing via GitHub PRs.

Reality I found: one dataset is clearly synthetic/broken (unrendered {product\_purchased} placeholders; satisfaction scores present for only 1/3 of rows); the two datasets aren't joinable (different domains).

My intended approach:

- Audit-first: prove the data-quality problems instead of hiding them ("I don't polish garbage").
- Frame it as a data PRODUCT: a medallion ETL (Bronze→Silver→Gold) that ingests the mess and outputs a qualified dataset, with data-quality gates (PASS/WARN/FAIL) that formally reject what's unrecoverable.
- Use generative AI only where it recovers real signal (reconstructing degraded ticket text, intent extraction, suggested replies).
- Deliver a polished standalone interactive HTML (touch the enriched data, watch the classifier run), with modular "ports" for future automation.
- Version everything (decision log + changelog) for full auditability.

Research questions — I want real evidence, not generic advice:

1. In real AI/data take-home hiring challenges, what specifically made WINNING submissions stand out vs. the average? Prioritize first-hand accounts from candidates or evaluators.
2. Does an "audit-first / refuse-to-fake-bad-data" stance help or hurt in evaluation? Any real accounts of candidates who flagged broken data?
3. Interactive product/app vs. written report/notebook — which lands better with evaluators, per actual reports?
4. How do strong candidates most commonly over-engineer and lose?

Cite sources. Favor first-hand accounts (Reddit, engineering blogs, hiring-manager write-ups) over SEO listicles.

The strongest take-home submissions usually win by reducing evaluator risk: they show sharp problem framing, make explicit tradeoffs, acknowledge gaps instead of hiding them, and leave behind something concrete that is easy to inspect quickly. In your case, the “audit-first, reject bad data, ship a usable productized pipeline plus demo” angle is directionally strong because it signals judgment under messy real constraints rather than blind compliance with a broken prompt. <sup>[1]</sup> <sup>[2]</sup> <sup>[3]</sup> <sup>[4]</sup>

## What stands out

Across first-hand or practitioner write-ups, standout candidates are the ones who understand the problem before building, defend decisions with evidence, and communicate their reasoning clearly rather than “proving” effort through sheer volume of work. Colin Breck explicitly says a take-home is valuable when it reveals deliberate, logical thinking, while Andrew Rondeau argues good evaluations focus on a small number of core concepts rather than sprawling output, and The Skip’s hiring-manager-oriented write-up says standout candidates first understood the problem before designing solutions. <sup>[2]</sup> <sup>[3]</sup> <sup>[1]</sup>

A recurring theme is inspectability: artifacts that make claims easy to verify tend to land better than raw cleverness. The “leave something behind” advice is basically “reduce hiring risk with proof,” and even Origin’s public data take-home asks candidates for both a public analysis project and a presentation, which implies evaluators value a package they can browse fast, not just code buried in a notebook. <sup>[5]</sup> <sup>[4]</sup>

## Audit-first stance

Your “refuse to fake bad data” stance is more likely to help than hurt if you pair it with recovery paths and formal decisioning instead of sounding combative. Multiple sources stress that strong candidates should acknowledge gaps, ask clarifying questions early, and frame assumptions explicitly; that maps well to an audit that says “this source passes/warns/fails, here is what is recoverable, here is what is rejected, here is why.” <sup>[6]</sup> <sup>[7]</sup> <sup>[3]</sup>

I did not find many clean first-hand stories specifically saying “I won because I flagged broken data,” but I did find adjacent evaluator guidance that rewards candidates who surface ambiguity instead of inventing certainty. Intrico’s summary of take-home advice emphasizes bringing the solution to life while also acknowledging gaps, and techinterview.guide recommends using early clarification questions when prompts are vague, which supports your idea of formally documenting data defects rather than smoothing them over. <sup>[7]</sup> <sup>[6]</sup>

## Demo vs. report

An interactive demo usually helps when it makes evaluation faster, not when it becomes a distraction. Public assignment instructions and hiring advice consistently favor a combination of artifacts—a browsable repo, a concise narrative, and something concrete to inspect—over a notebook-only submission, so a polished standalone HTML plus decision log is likely stronger than a report alone as long as the demo exposes the logic rather than hiding it behind UI. <sup>[4]</sup> <sup>[5]</sup>

The risk is that a flashy app can backfire if evaluators cannot quickly map it to the challenge criteria. The strongest pattern from the sources is “make the thinking legible”: use the demo as

a front-end to your audit, ETL gates, and AI-assisted recovery workflow, then back it with a short written readme that states dataset defects, acceptance criteria, and what the prototype proves. <sup>[3]</sup> <sup>[1]</sup> <sup>[2]</sup>

## How candidates lose

The most common failure mode is over-engineering beyond the signal the assignment can actually measure. Rondeau argues good take-homes should target only a few critical concepts, Elena Dyachkova says take-homes should be well-defined and reasonable in 1–2 hours from the employer side, and multiple Reddit threads reflect candidate frustration when assignments balloon into mini-products—suggesting evaluators often do not reward proportional extra effort. <sup>[8]</sup> <sup>[9]</sup> <sup>[2]</sup>

In practical terms, over-engineering usually looks like:

- Building infrastructure the prompt did not justify, instead of surfacing the key business constraint. <sup>[2]</sup> <sup>[3]</sup>
- Hiding uncertainty behind polished outputs instead of naming assumptions and limits. <sup>[6]</sup> <sup>[3]</sup>
- Spending time on completeness rather than inspectability, making reviewers dig for the real insight. <sup>[1]</sup> <sup>[4]</sup>
- Treating the submission like unpaid production work rather than a focused demonstration of judgment. <sup>[8]</sup> <sup>[2]</sup>

## Best play for your case

Based on the evidence, your highest-upside submission is probably: 1) a blunt but professional audit proving why the raw datasets are not trustworthy as-is, 2) a medallion pipeline with explicit PASS/WARN/FAIL quality gates, 3) limited AI use only where it recovers signal, and 4) a lightweight interactive artifact that lets reviewers inspect records, defects, transformations, and model behavior in minutes. That package aligns with what evaluator-oriented sources reward most—problem understanding, evidence-backed tradeoffs, and concrete proof—while avoiding the classic “I built too much around bad premises” trap. <sup>[5]</sup> <sup>[3]</sup> <sup>[4]</sup> <sup>[1]</sup> <sup>[2]</sup>

A good framing line for the PR is: the winning move was not to pretend the brief was clean, but to turn an unreliable prompt into an auditable customer-support data product with explicit failure handling. Would you like a PR structure next—README outline, repo tree, and the exact narrative to make this beat the baseline?

\*~\*

1. <https://blog.colinbreck.com/an-interview-process-that-works-for-me/>
2. <https://blog.andrewrondeau.com/interviewing/software/careers/2020/08/04/take-home-vs-whiteboard-coding-the-pr-blem-is-bad-interviews.html>
3. <https://theskip.substack.com/p/what-pm-hiring-managers-actually>
4. <https://artoffindingwork.substack.com/p/when-interviewing-leave-something>

5. <https://github.com/OriginFinancial/data-take-home-assignment>
6. <https://intrico.io/interview-best-practices/the-dreaded-take-home-assignment>
7. <https://techinterview.guide/sections/interviewing/tech-challenges/unmonitored-challenges>
8. <https://dramaticanalyst.substack.com/p/the-ring-of-hire-part-1>
9. [https://www.reddit.com/r/recruitinghell/comments/11sy12d/rejected\\_after\\_take\\_home\\_assignment\\_i\\_spent\\_20/](https://www.reddit.com/r/recruitinghell/comments/11sy12d/rejected_after_take_home_assignment_i_spent_20/)
10. [https://www.reddit.com/r/ExperiencedDevs/comments/1htzvzp/has\\_anyone\\_ever\\_been\\_paid\\_for\\_a\\_takehome/](https://www.reddit.com/r/ExperiencedDevs/comments/1htzvzp/has_anyone_ever_been_paid_for_a_takehome/)
11. <https://www.imocha.io/blog/take-home-assignment-interview>
12. [https://www.linkedin.com/posts/kristincrosier\\_dear-hiring-managers-please-stop-including-activity-7374485784374124545-iNHa](https://www.linkedin.com/posts/kristincrosier_dear-hiring-managers-please-stop-including-activity-7374485784374124545-iNHa)
13. <https://dev.to/rockarts/take-home-assignments-and-the-interview-process-4jhl>
14. <https://www.indeed.com/hire/c/info/take-home-interview-assignment>
15. <https://fabrichq.ai/blogs/how-ai-cheating-killed-take-home-assignments>
16. <https://news.ycombinator.com/item?id=15553961>
17. <https://www.youtube.com/watch?v=9ablKbCxZWE>
18. [https://www.reddit.com/r/ExperiencedDevs/comments/1klvv0n/hiring\\_managers\\_what\\_do\\_you\\_hate\\_about\\_takehome/](https://www.reddit.com/r/ExperiencedDevs/comments/1klvv0n/hiring_managers_what_do_you_hate_about_takehome/)
19. <https://towardsdatascience.com/please-stop-giving-take-home-assignments-to-data-science-candidates-e47b1cc5a9f3/>
20. <https://www.youtube.com/watch?v=sgmMqP9AYpY>
21. <https://www.reforge.com/artifacts/c/hiring/take-home-assignment>
22. [https://www.reddit.com/r/recruitinghell/comments/1bo3l9d/can\\_we\\_all\\_collectively\\_just\\_say\\_no\\_to\\_the\\_take/](https://www.reddit.com/r/recruitinghell/comments/1bo3l9d/can_we_all_collectively_just_say_no_to_the_take/)
23. [https://www.reddit.com/r/gamedev/comments/o101la/a\\_5day\\_takehome\\_assignment\\_before\\_the\\_first/](https://www.reddit.com/r/gamedev/comments/o101la/a_5day_takehome_assignment_before_the_first/)
24. [https://www.reddit.com/r/dataengineering/comments/10qdxjr/feel\\_a\\_bit\\_burned\\_from\\_a\\_takehome\\_assignment\\_i/](https://www.reddit.com/r/dataengineering/comments/10qdxjr/feel_a_bit_burned_from_a_takehome_assignment_i/)
25. [https://www.reddit.com/r/cscareerquestions/comments/nmphcg/pet\\_peek\\_this\\_takehome\\_project\\_should\\_take\\_2/](https://www.reddit.com/r/cscareerquestions/comments/nmphcg/pet_peek_this_takehome_project_should_take_2/)
26. [https://www.reddit.com/r/cscareerquestions/comments/ffq25a/falsely\\_flagged\\_for\\_plagiarism\\_by\\_hackerrank/](https://www.reddit.com/r/cscareerquestions/comments/ffq25a/falsely_flagged_for_plagiarism_by_hackerrank/)
27. [https://www.reddit.com/r/cscareerquestions/comments/bjknqg/getting\\_constant\\_take\\_home\\_coding\\_projects\\_for/](https://www.reddit.com/r/cscareerquestions/comments/bjknqg/getting_constant_take_home_coding_projects_for/)
28. [https://www.reddit.com/r/recruitinghell/comments/ochzho/the\\_take\\_home\\_assignment\\_i\\_did\\_was\\_too\\_good\\_i/](https://www.reddit.com/r/recruitinghell/comments/ochzho/the_take_home_assignment_i_did_was_too_good_i/)
29. [https://www.reddit.com/r/webdev/comments/1jwckdw/q\\_for\\_those\\_ranting\\_about\\_leetcode\\_take\\_home/](https://www.reddit.com/r/webdev/comments/1jwckdw/q_for_those_ranting_about_leetcode_take_home/)
30. [https://www.reddit.com/r/jobhunting/comments/1lklb66/that\\_white\\_text\\_resume\\_hack\\_is\\_complete\\_bs\\_and/](https://www.reddit.com/r/jobhunting/comments/1lklb66/that_white_text_resume_hack_is_complete_bs_and/)
31. <https://blog.urth.org/2017/07/14/a-technical-hiring-process/>
32. [https://www.reddit.com/r/golang/comments/1coqhsa/rejected\\_after\\_golang\\_take\\_home\\_assignment\\_any/](https://www.reddit.com/r/golang/comments/1coqhsa/rejected_after_golang_take_home_assignment_any/)
33. [https://www.reddit.com/r/GradSchool/comments/1jx2urk/i\\_was\\_flagged\\_by\\_turnitins\\_ai\\_detector\\_now\\_my/](https://www.reddit.com/r/GradSchool/comments/1jx2urk/i_was_flagged_by_turnitins_ai_detector_now_my/)
34. <https://productgrindhq.substack.com/p/the-2026-pm-job-market-patterns-from>
35. <https://debliu.substack.com/p/how-to-land-at-a-fast-paced-startup>
36. <https://resonancesearch.substack.com/p/16-ways-to-stand-out-in-a-hiring>
37. [https://www.reddit.com/r/ExperiencedDevs/comments/11tbgfd/take\\_home\\_assignment\\_for\\_senior\\_role\\_rejected/](https://www.reddit.com/r/ExperiencedDevs/comments/11tbgfd/take_home_assignment_for_senior_role_rejected/)
38. <https://josvisser.substack.com/p/interviewing-has-become-a-mess>
39. [https://www.reddit.com/r/dataengineering/comments/165fn08/got\\_feedback\\_for\\_take\\_home\\_project\\_rejected/](https://www.reddit.com/r/dataengineering/comments/165fn08/got_feedback_for_take_home_project_rejected/)

40. <https://muriyahfraser.substack.com/p/i-received-12000-applications-for>
41. [https://www.reddit.com/r/cscareerquestions/comments/1it1o1h/rejected a 1 week take home assignment/](https://www.reddit.com/r/cscareerquestions/comments/1it1o1h/rejected_a_1_week_take_home_assignment/)
42. <https://earlyandexec.substack.com/p/early-a-special-edition-on-jack-and>
43. [https://www.reddit.com/r/cscareerquestions/comments/1ecem9q/anyone refuse to do take home assignments/](https://www.reddit.com/r/cscareerquestions/comments/1ecem9q/anyone_refuse_to_do_take_home_assignments/)
44. <https://alexoppenheimer.substack.com/p/getting-hired-and-hiring-are-both>